Perceptron & Neural Nets

Raffaele Scaringi

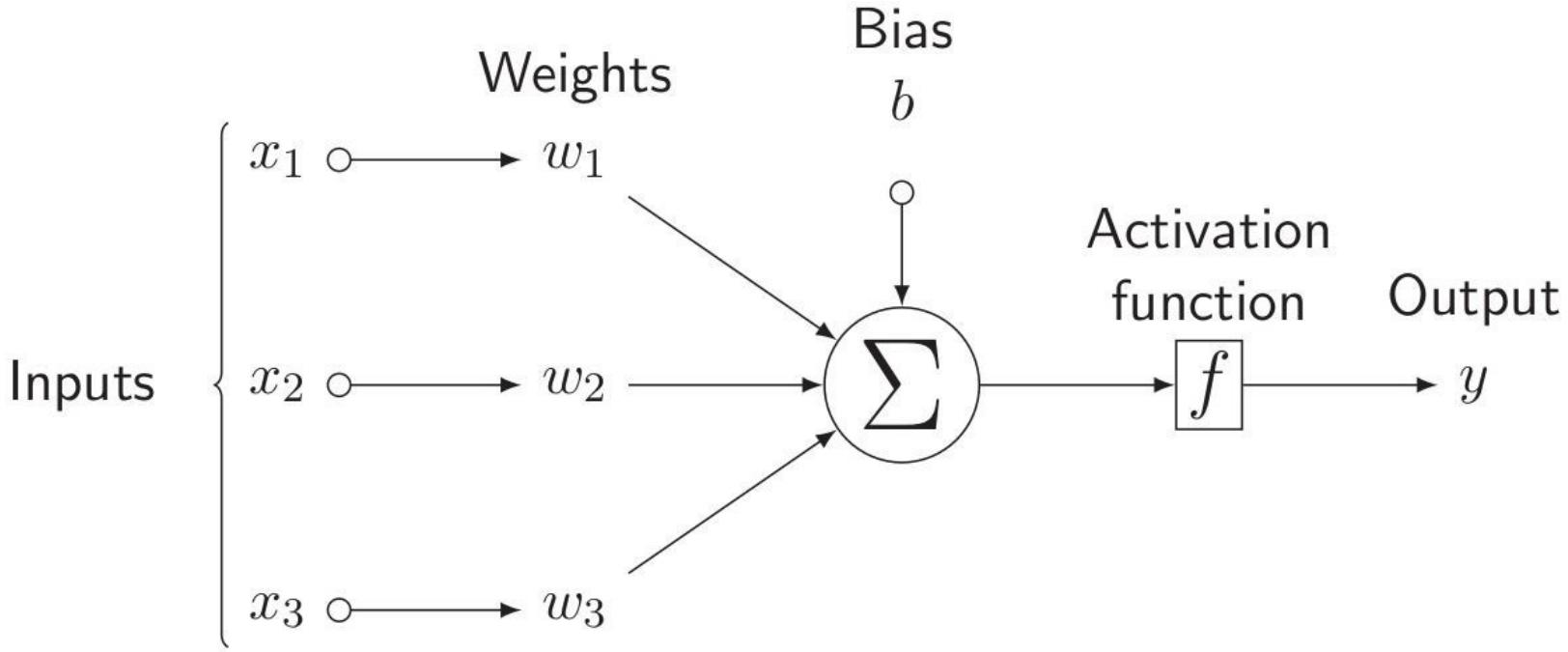
Agenda

1) Perceptron
   - Intro to optimization
   - Perceptron
   - Optimizers
   - Hands-on tutorial

2) Feedforward Neural Networks
   - The limitations of Perceptrons
   - Multi-layer Perceptron
   - Training: the forward and back-propagation
   - Debugging tips

Perceptron: Threshold Logic

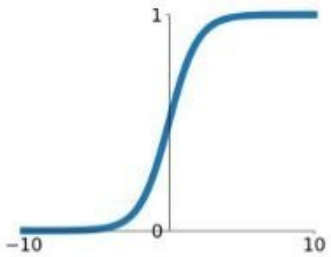
$$\mathcal{L}_{\text{perc}}(\mathbf{x}, y) = \begin{cases} 0 & \text{if } y\mathbf{w}^\top \mathbf{f}(\mathbf{x}) > 0 \\ -y\mathbf{w}^\top \mathbf{f}(\mathbf{x}) & \text{if } y\mathbf{w}^\top \mathbf{f}(\mathbf{x}) \leq 0 \end{cases}$$



Activation functions

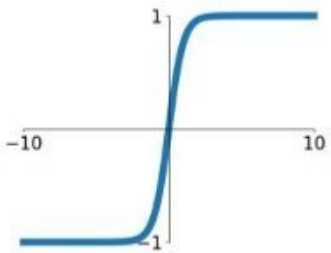
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



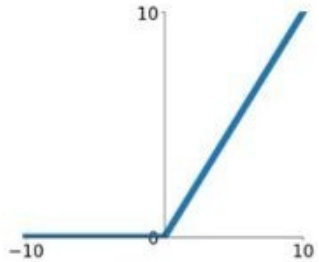
tanh

$$\tanh(x)$$



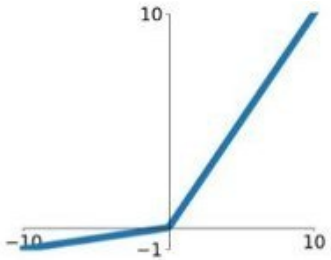
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

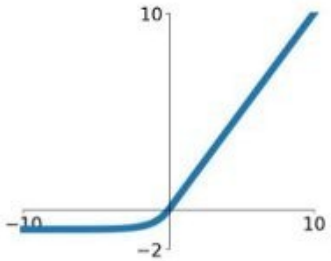


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Optimizers (pt1)

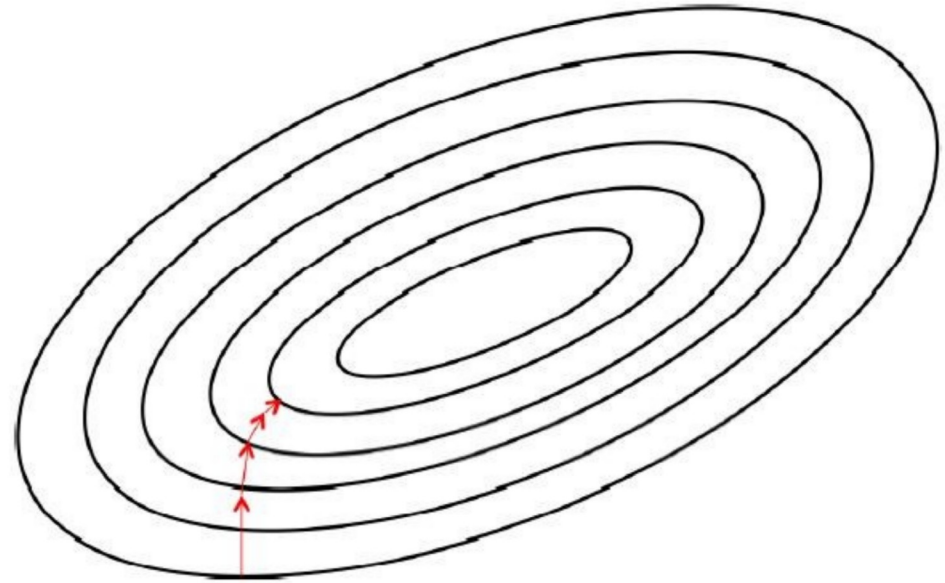
Gradie
nt

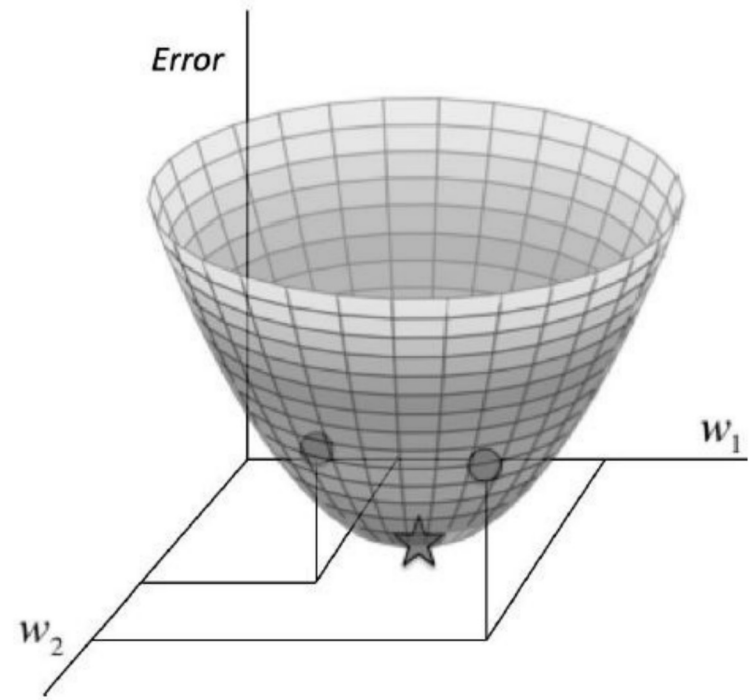
$$\Delta w_k = -\frac{\partial E}{\partial w_k}$$

$$= -\frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + \Delta w_k$$

Stochastic gradient descent (**SGD**)





Optimizers (pt1)

Hyperparameters

- Learning rate (α)

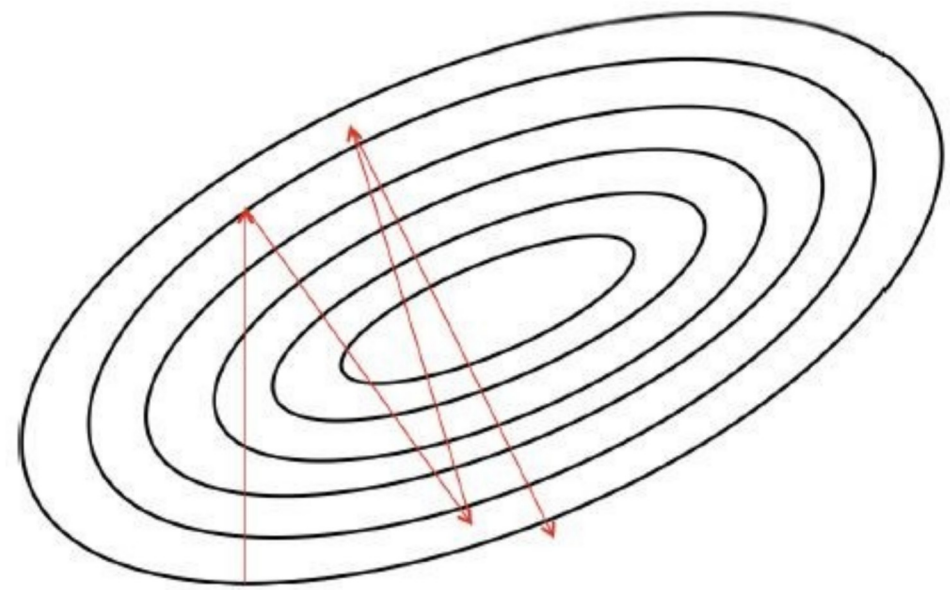
$$\Delta w_k = -\alpha \frac{\partial E}{\partial w_k}$$

$$= -\alpha \frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + \Delta w_k$$

Stochastic gradient descent (**SGD**)

Practical test:
lr_val = [1; 0.1; 0.01]
momentum_val = 0 nesterov_val = ‘False' decay_val = 1e-6



Result of a large learning rate α

Optimizers

Hyperparameters

- Learning rate (α)

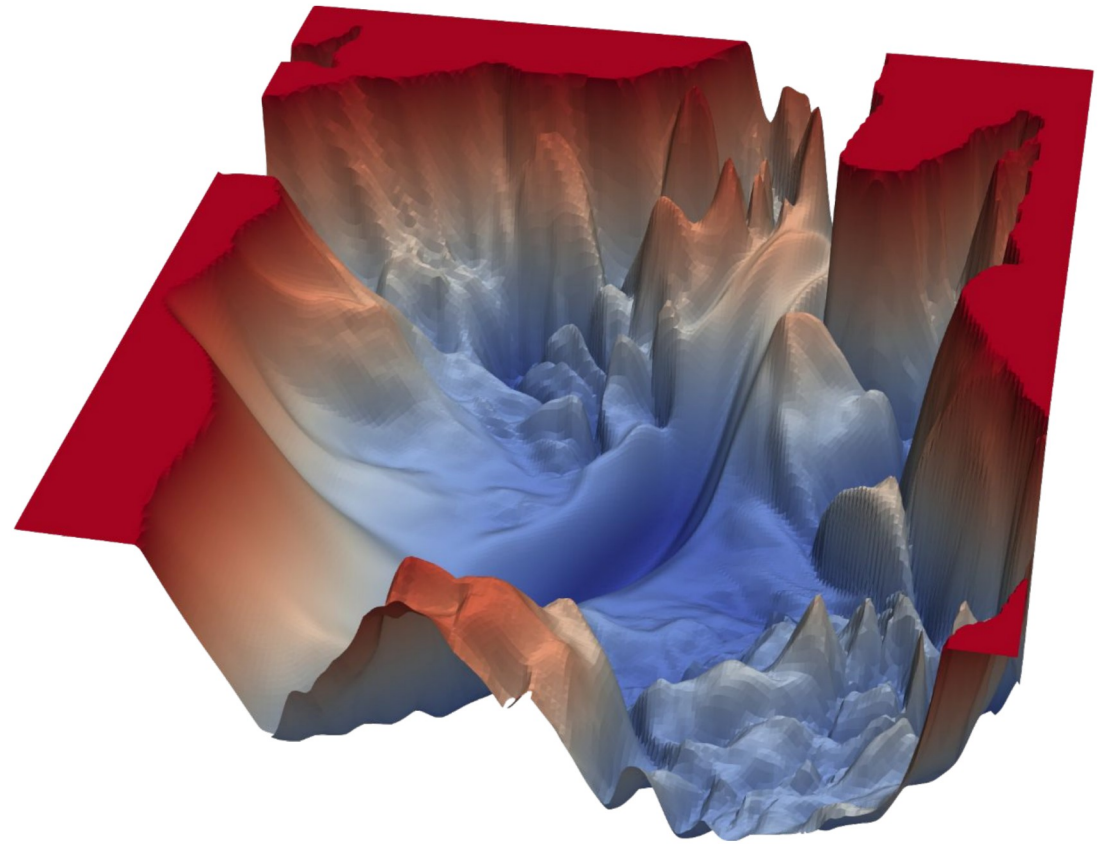
$$\Delta w_k = -\alpha \frac{\partial E}{\partial w_k}$$

$$= -\alpha \frac{\partial}{\partial w_k} \left(\frac{1}{m} \sum_i (w^T X_i - y_i)_i^2 \right)$$

$$w_{i+1} = w_i + \Delta w_k$$

Stochastic gradient descent (**SGD**)

Watch out for local minimal areas



Optimizers

Optimizers

Hyperparameters

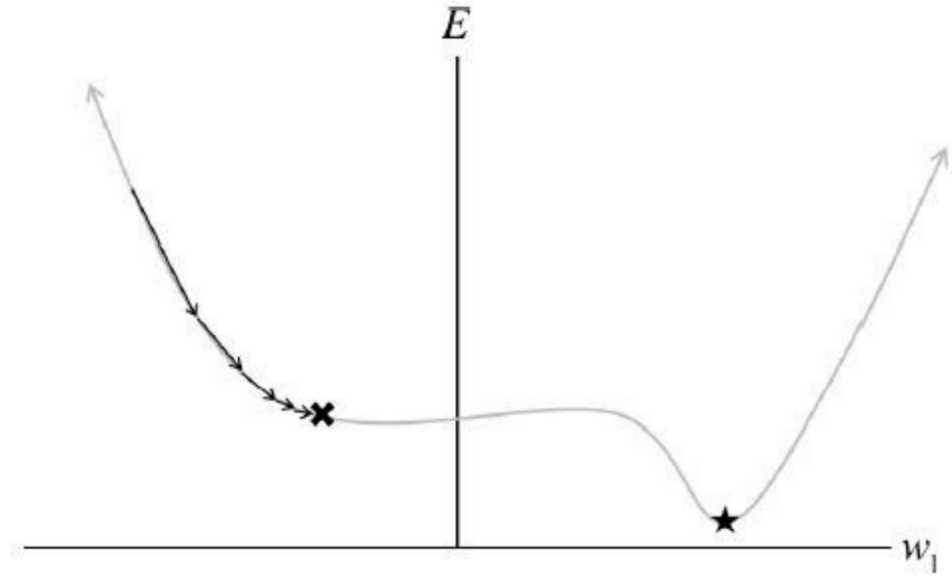
- Learning rate (α)

$$\Delta w_k = -\alpha \frac{\partial E}{\partial w_k}$$

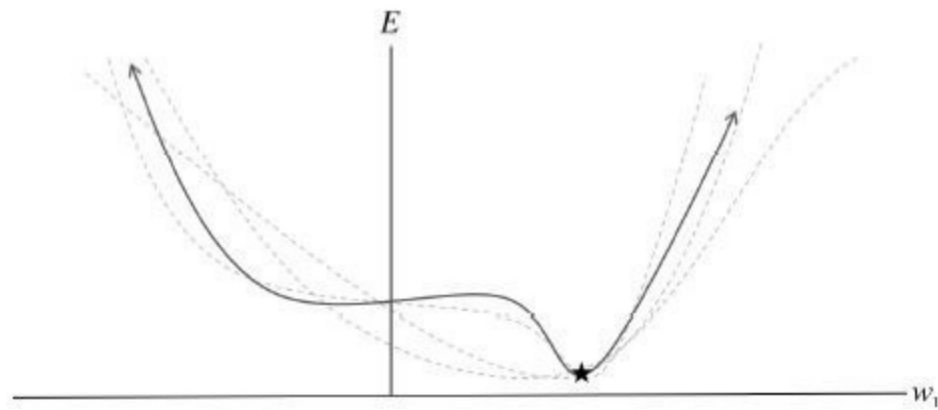
$$= -\alpha \frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + \Delta w_k$$

Stochastic gradient descent (**SGD**)



Local Minima



Multiple samples

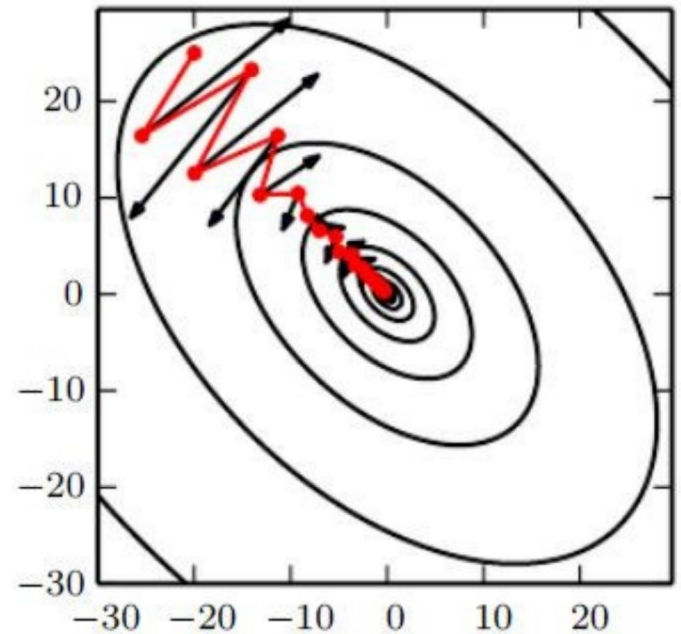
Optimizers

Hyperparameters

- Learning rate (α)
- Momentum (β)

$$v_{i+1} = v\beta - \alpha \frac{\partial}{\partial w_k}\left(\frac{1}{m}\sum_i (w^T X_i - y_i)_i^2\right)$$

$$w_{i+1} = w_i + v$$



SGD

SGD+Momentum

Stochastic gradient descent with momentum (**SGD+Momentum**)

Optimizers

Hard to pick right hyperparameters

- Small learning rate: long convergence time
- Large learning rate: convergence problems

$$\Delta w_{k,t} = -\alpha \frac{\partial E_t}{\partial w_{k,t}} = -\alpha \nabla_w E(w_t)$$

Adagrad: adapts learning rate to each parameter

$$g_{t,i} = \nabla_w E(w_{t,i})$$

$$G_{t+1,i} = G_{t,i} + g_{t,i} \odot g_{t,i}$$

accumulated squared



$$w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{G_{t,i} + \epsilon}} g_{t,i}$$

- Learning rate might decrease too fast
- Might not converge

Optimizers

ADAM: decaying average of the past squared gradients and momentum

RMSprop / Adadelta

$$g_{t,i} = \nabla_w E(w_{t,i})$$

$$G_{t+1,i} = \gamma G_{t,i} + (1-\gamma) g_{t,i} \odot g_{t,i}$$

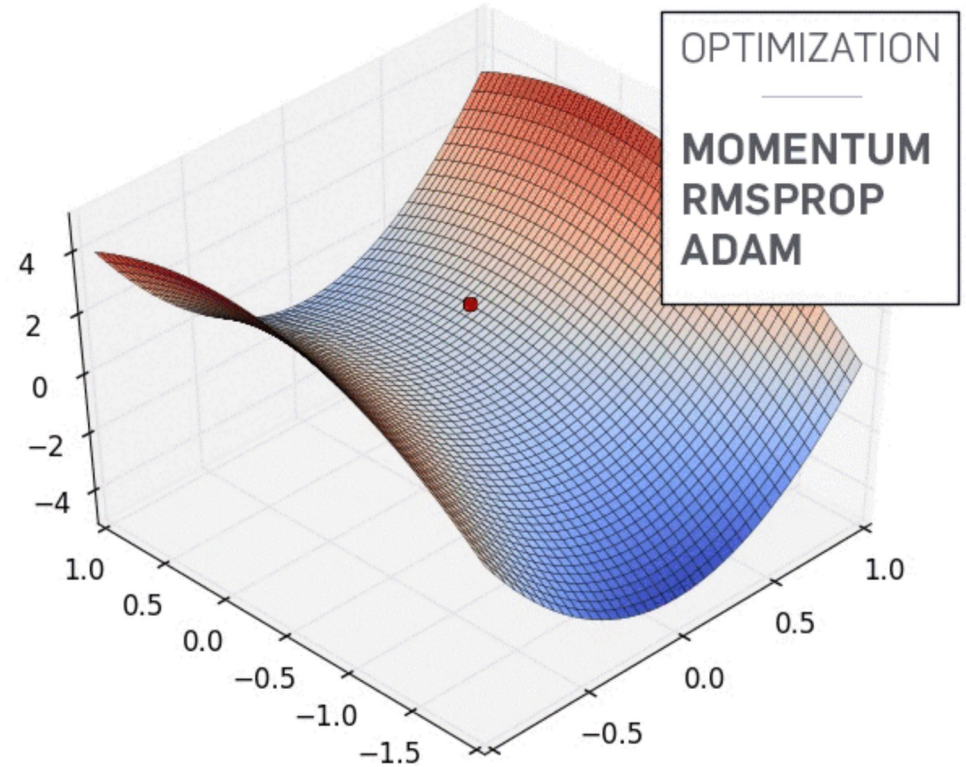


$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \qquad \hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$w_{t+1,i} = w_{t,i} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

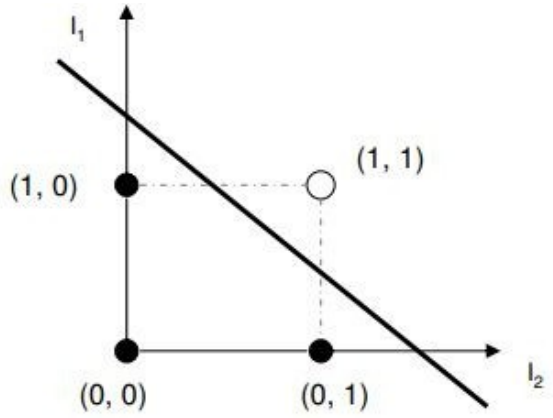
Which optimizer is the best?



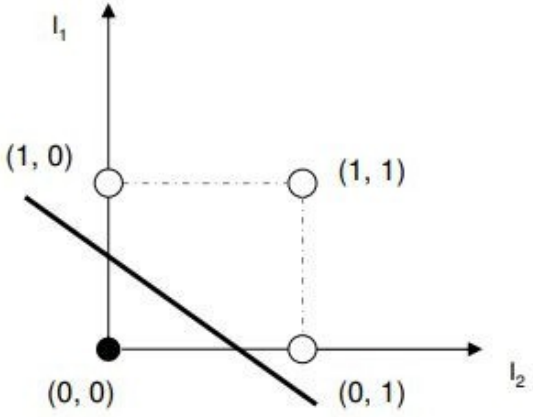
Multi-layer Perceptron

Limitations of the Perceptron

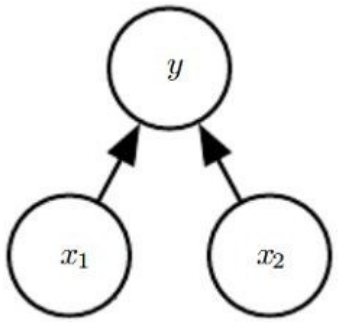
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1

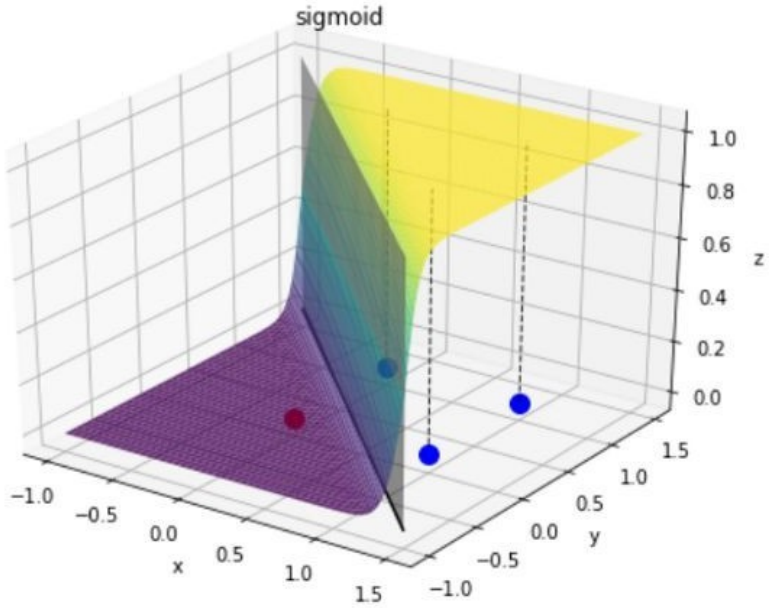


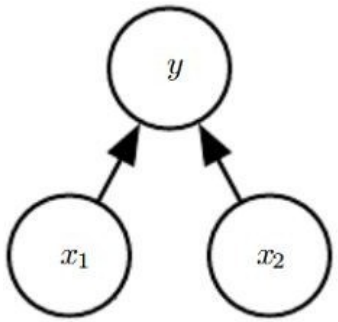
OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

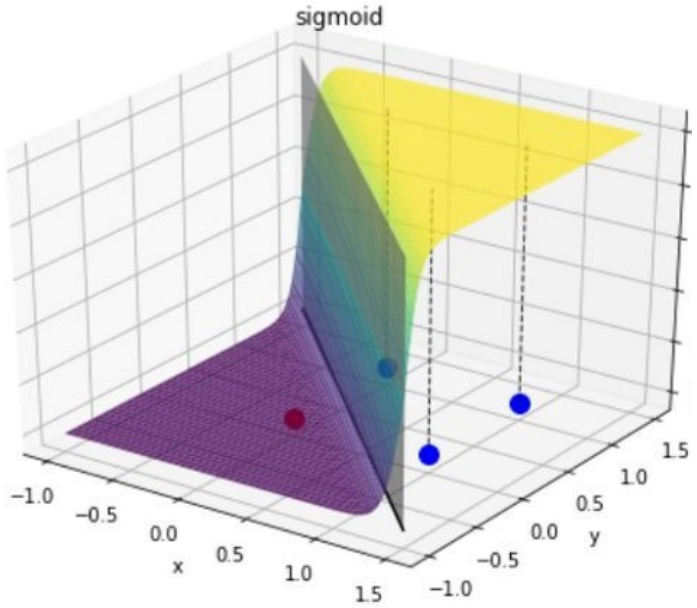


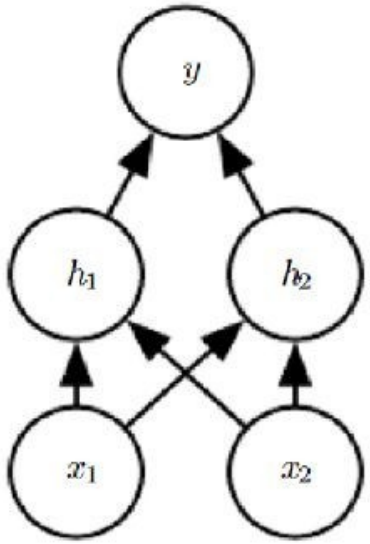
Perceptron


y
x_1
x_2

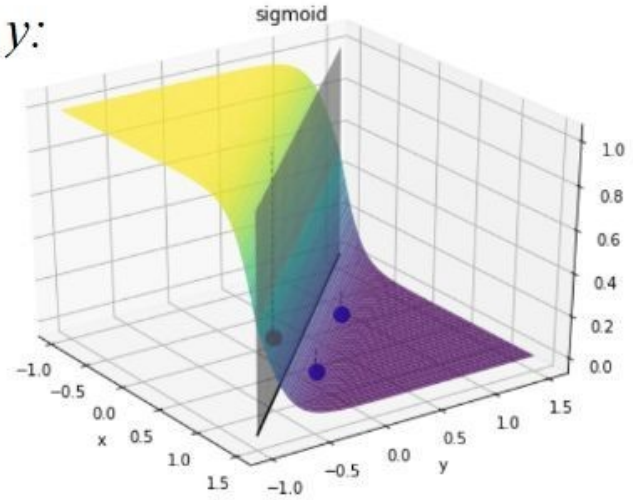

sigmoid
1.0
0.8
0.6
0.4
0.2
0.0
z
−1.0
−0.5
0.0
0.5
1.0
1.5
x
−1.0
−0.5
0.0
0.5
1.0
1.5
y


y
x_1
x_2

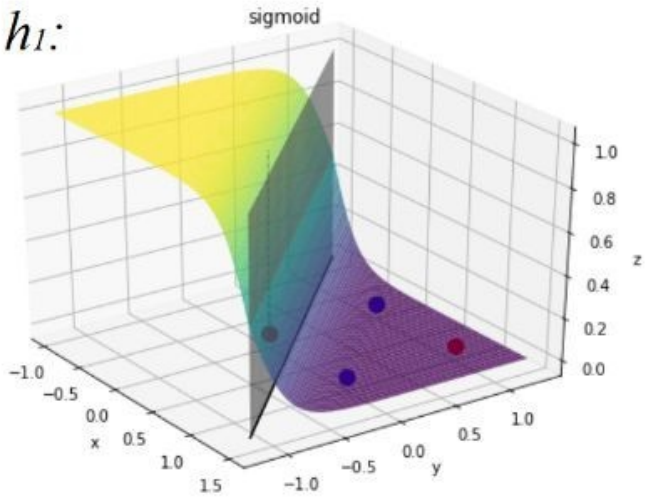

sigmoid
x
y


y
h_1
h_2
x_1
x_2

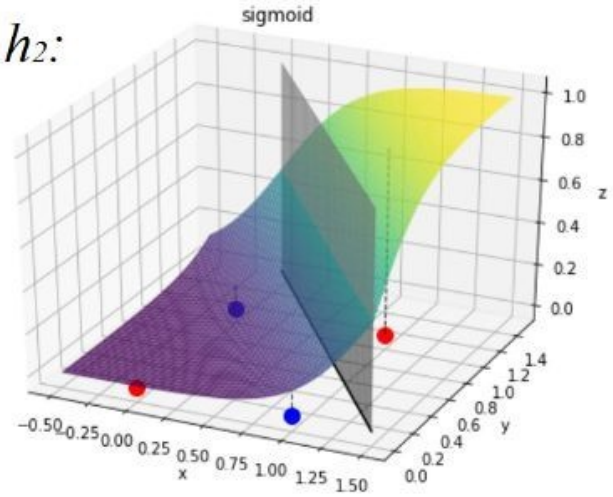
y:

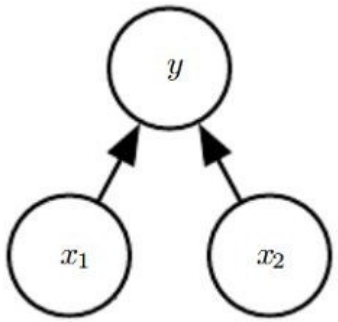

sigmoid
x
y

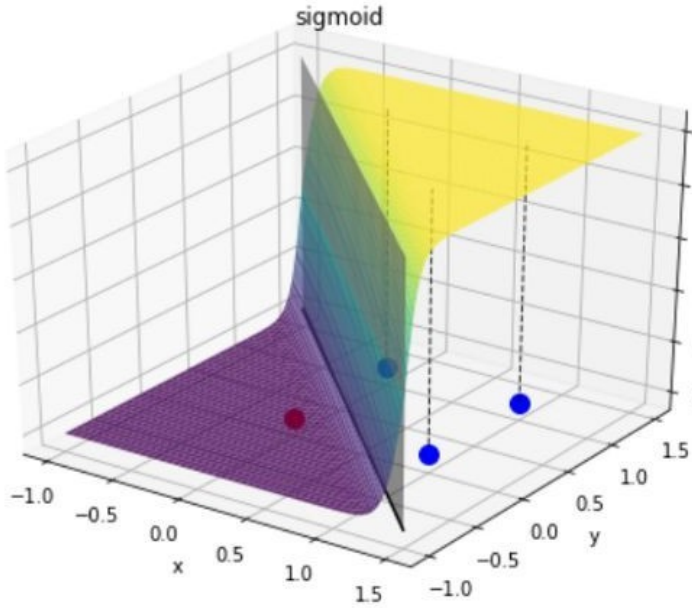
h_1:

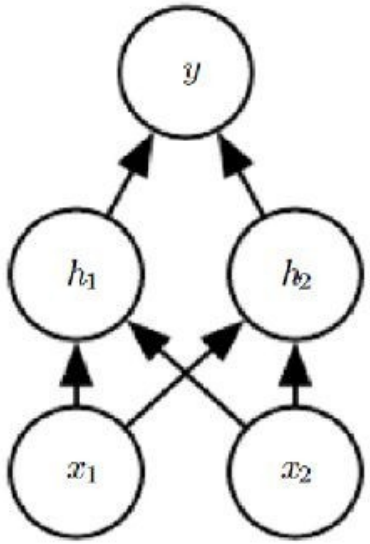

sigmoid
x
y
z

h_2:

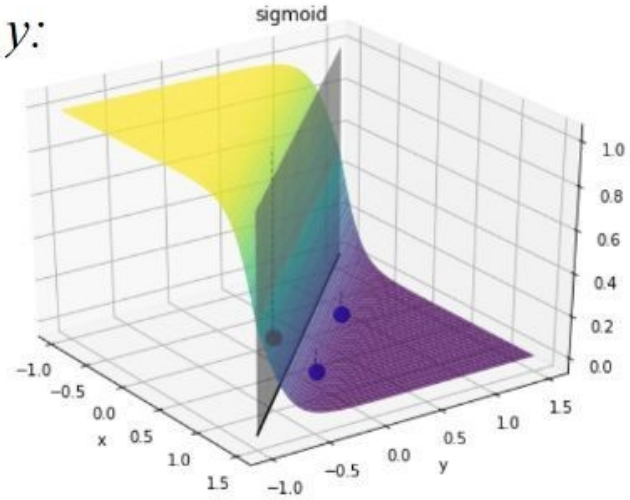

sigmoid
x
y
z

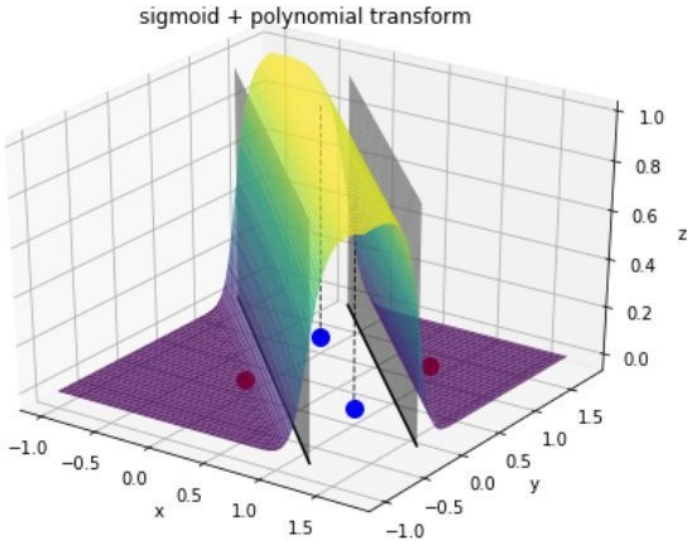

y
x_1
x_2


sigmoid
x
y

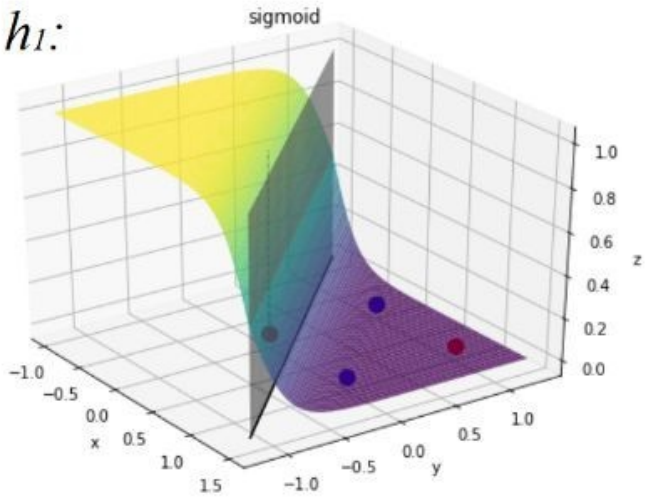

y
h_1
h_2
x_1
x_2

y:

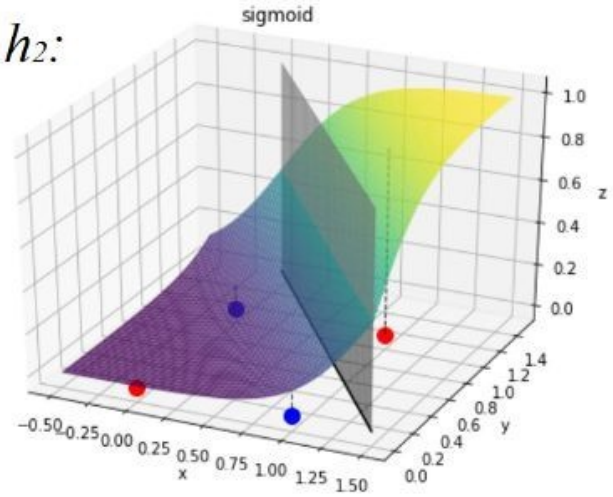

sigmoid
x
y


sigmoid + polynomial transform
x
y
z

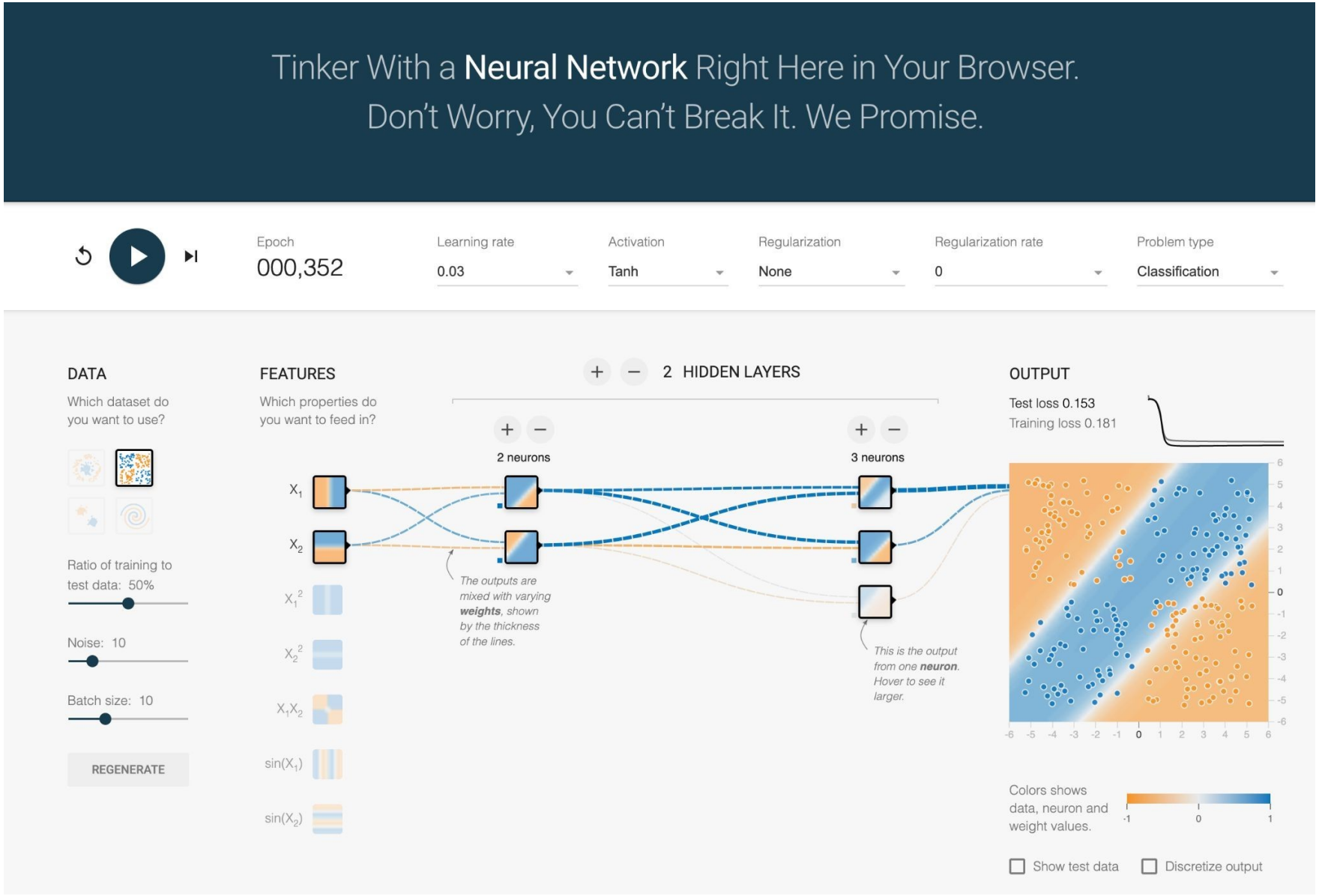
h_1:


sigmoid
x
y
z

h_2:

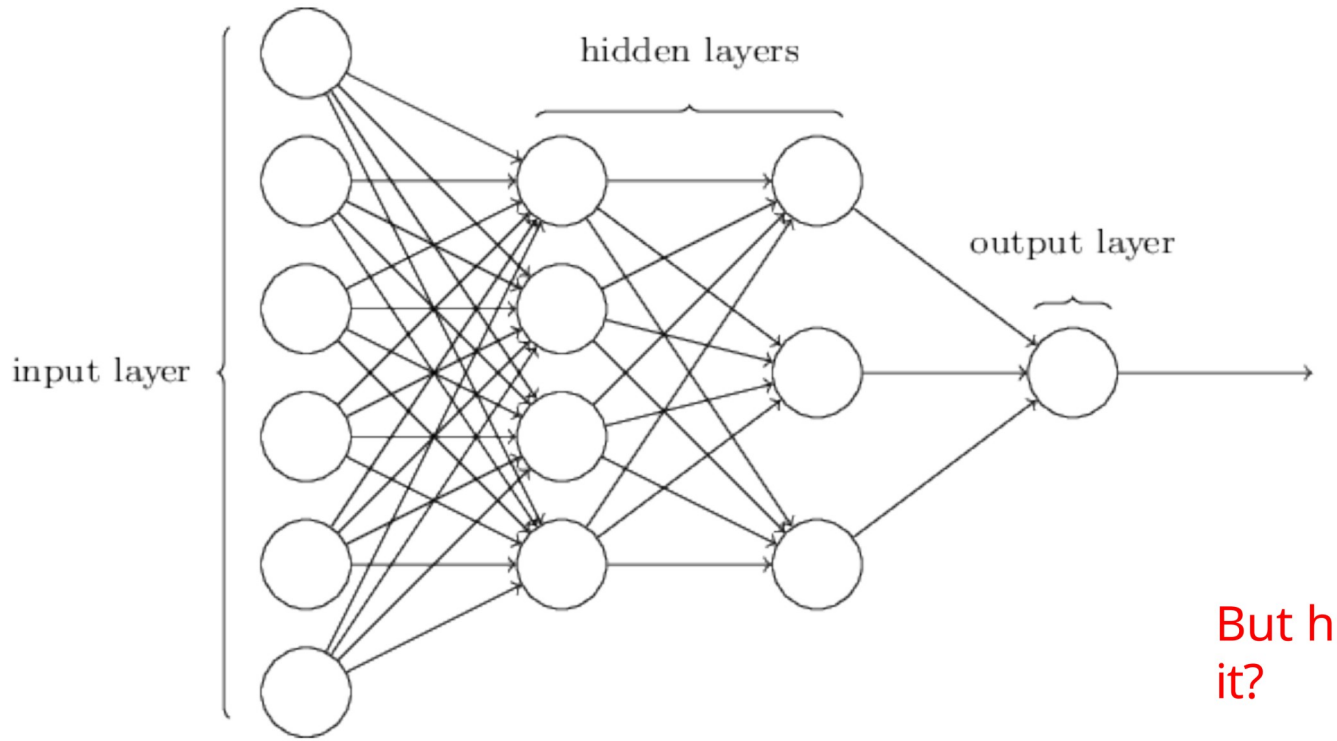

sigmoid
x
y
z

Let’s play with it!



Try it [here](#)

Architecture of Neural Networks



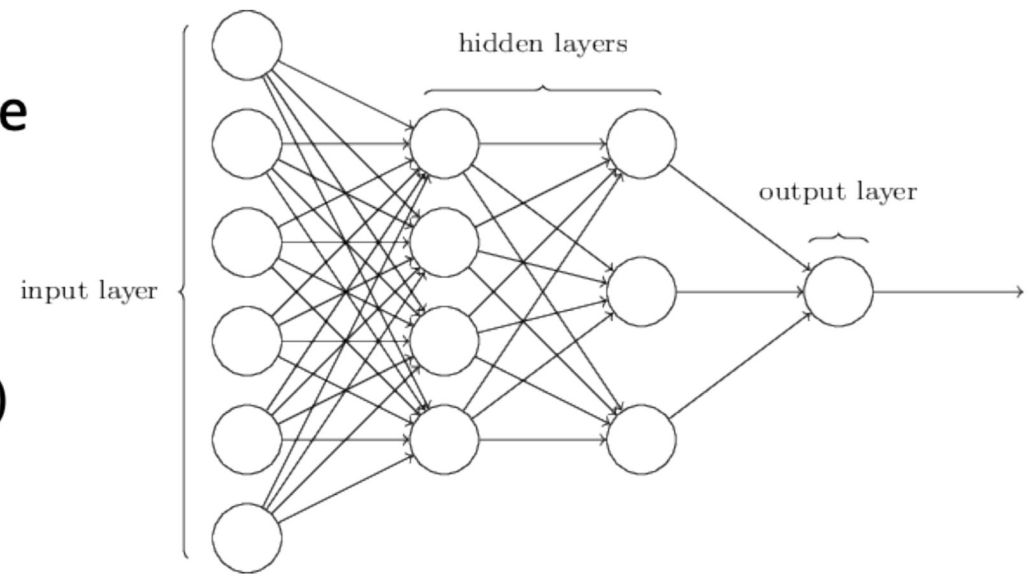
But how do we train it?

- Sometimes called multi-layer perceptron (MLP)
- Output from one layer is used as input for the next (Feedforward network)

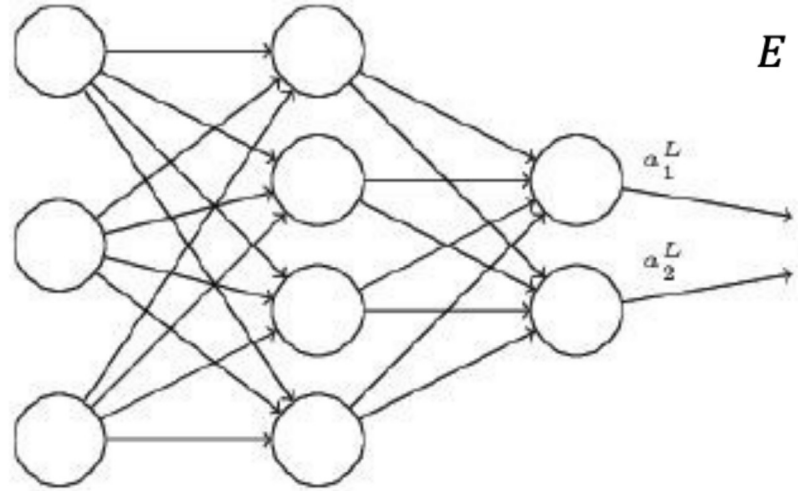
Forward Propagation

- Store weights and biases as matrices
- Suppose we are considering the weights from the second (hidden) layer to the third (output) layer
  - w is the weight matrix with w_{ji} the weight for the connection between the ith neuron in the second layer and the jth neuron in the third layer
  - b is the vector of biases in the third layer
  - a is the vector of activations (output) of the 2nd layer
  - a' the vector of activations (output) of the third layer

$$\boldsymbol{a' = \sigma(wa + b)}$$



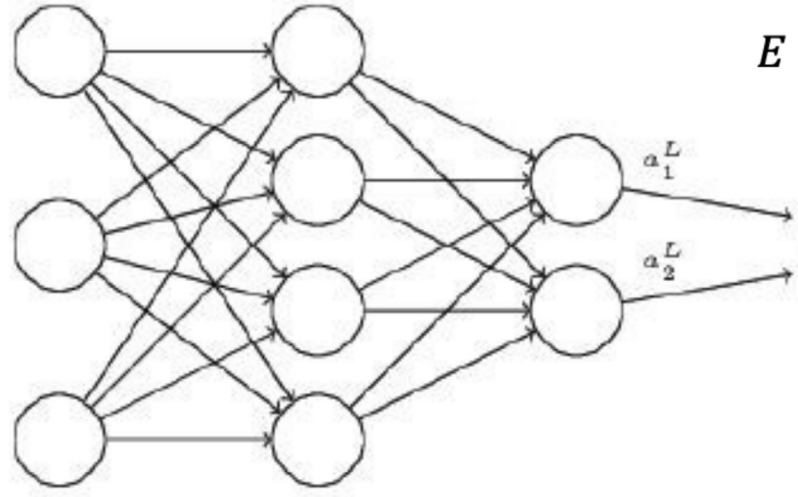
Backpropagation



$$E = \frac{1}{2}\sum_i \left(a_i^L - y_i\right)^2$$

$$cost\ E = E(a^L)$$

Backpropagation

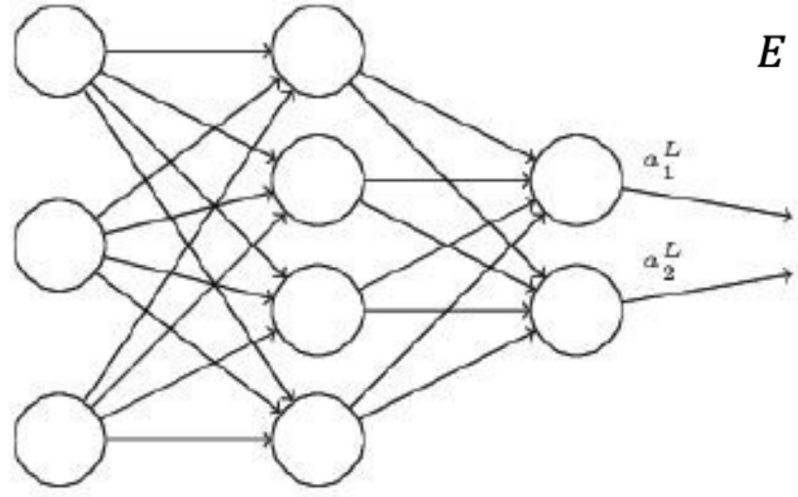


$$E = \frac{1}{2}\sum_i \left(a_i^L - y_i\right)^2$$

$$cost\ E = E(a^L)$$

$$z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l \qquad a_j^l = \sigma\left(\sum_i w_{ji}^l a_i^{l-1} + b_j^l\right) = \sigma(z_j^l)$$

Backpropagation



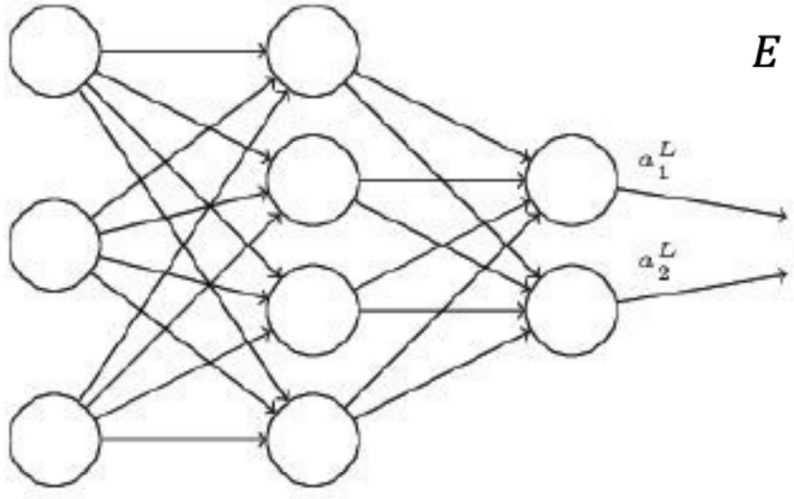
$$E = \frac{1}{2}\sum_i \left(a_i^L - y_i\right)^2$$

$$cost\ E = E(a^L)$$

$$z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l \qquad a_j^l = \sigma\left(\sum_i w_{ji}^l a_i^{l-1} + b_j^l\right) = \sigma(z_j^l)$$

$$\delta_j^L \equiv \frac{\partial E}{\partial z_j^L} = \frac{\partial E}{\partial a_i^L}\frac{\partial a_i^L}{\partial z_j^L} = \frac{\partial E}{\partial a_j^L}\sigma'(z_j^L) \qquad (1)$$

Backpropagation



$$E = \frac{1}{2}\sum_i \left(a_i^L - y_i\right)^2$$

$$cost\ E = E(a^L)$$

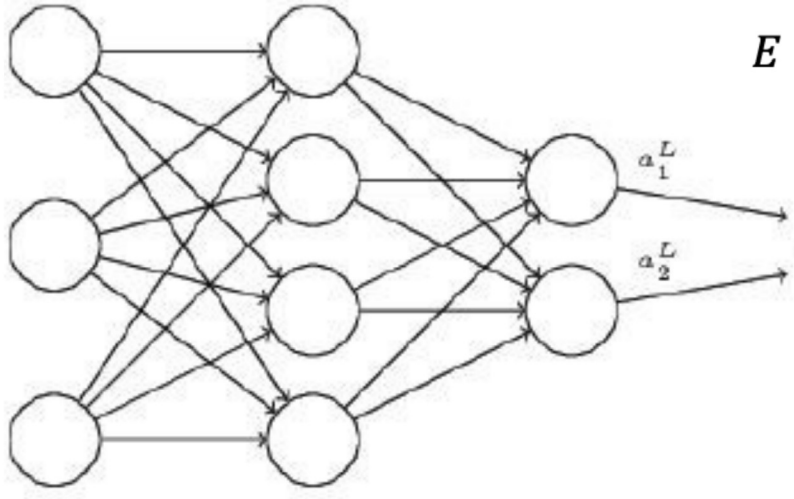
$$z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l \qquad a_j^l = \sigma\left(\sum_i w_{ji}^l a_i^{l-1} + b_j^l\right) = \sigma(z_j^l)$$

$$\delta_j^L \equiv \frac{\partial E}{\partial z_j^L} = \frac{\partial E}{\partial a_i^L}\frac{\partial a_i^L}{\partial z_j^L} = \frac{\partial E}{\partial a_j^L}\sigma'(z_j^L) \qquad (1)$$

$$\delta_j^l \equiv \frac{\partial E}{\partial z_j^l} = \frac{\partial E}{\partial z_i^{l+1}}\frac{\partial z_i^{l+1}}{\partial z_j^l} = \frac{\partial z_i^{l+1}}{\partial z_j^l}\delta_i^{l+1}$$

$$= \frac{\partial(\sum_i w_{ij}^{l+1} a_j^l + b_i^{l+1})}{\partial z_j^l}\delta_j^{l+1} = \sum_i w_{ij}^{l+1}\delta_i^{l+1}\sigma'(z_j^l) \qquad (2)$$

Backpropagation



$$E = \frac{1}{2}\sum_i \left(a_i^L - y_i\right)^2$$

$$cost\ E = E(a^L)$$

$$z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l \qquad a_j^l = \sigma\left(\sum_i w_{ji}^l a_i^{l-1} + b_j^l\right) = \sigma(z_j^l)$$

$$\delta_j^L \equiv \frac{\partial E}{\partial z_j^L} = \frac{\partial E}{\partial a_i^L}\frac{\partial a_i^L}{\partial z_j^L} = \frac{\partial E}{\partial a_j^L}\sigma'(z_j^L) \qquad (1)$$

$$\delta_j^l \equiv \frac{\partial E}{\partial z_j^l} = \frac{\partial E}{\partial z_i^{l+1}}\frac{\partial z_i^{l+1}}{\partial z_j^l} = \frac{\partial z_i^{l+1}}{\partial z_j^l}\delta_i^{l+1}$$

$$= \frac{\partial\left(\sum_i w_{ij}^{l+1} a_j^l + b_i^{l+1}\right)}{\partial z_j^l}\delta_j^{l+1} = \sum_i w_{ij}^{l+1}\delta_i^{l+1}\sigma'(z_j^l) \qquad (2)$$

$$\frac{\partial E}{\partial w_{ji}^l} = \frac{\partial E}{\partial a_i^l}\frac{\partial a_i^l}{\partial z_j^l}\frac{\partial (w_{ji}^l a_i^{l-1})}{\partial w_{ji}^l}$$

Backpropagation

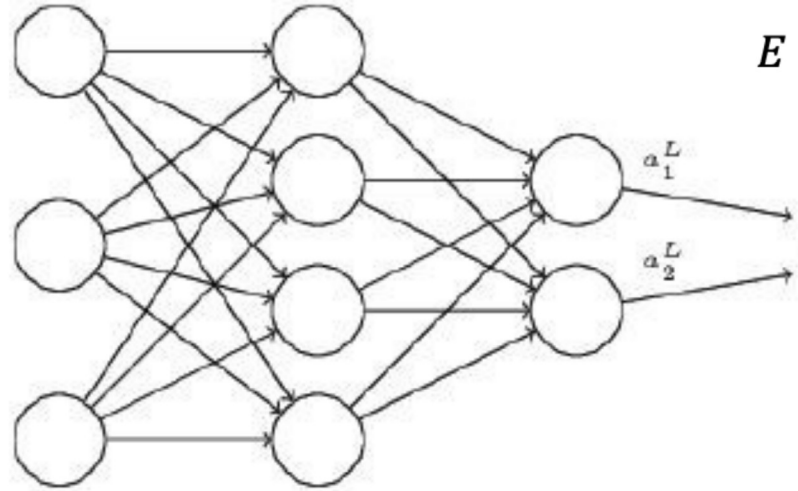
1. **Input x:** Set the corresponding activation a^1 for the input layer.

2. **Feedforward:** For each $l = 2, 3, \ldots, L$ compute $z^l = w^l a^{l-1} + b^l$ and $a^l = \sigma(z^l)$.

3. **Output error δ^L:** Compute the vector $\delta^L = \nabla_a C \odot \sigma'(z^L)$.

4. **Backpropagate the error:** For each $l = L-1, L-2, \ldots, 2$ compute $\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l)$.

5. **Output:** The gradient of the cost function is given by $\frac{\partial C}{\partial w^l_{jk}} = a^{l-1}_k \delta^l_j$ and $\frac{\partial C}{\partial b^l_j} = \delta^l_j$.



$$E = \frac{1}{2}\sum_i (a_i^L - y_i)^2$$

$$cost\ E = E(a^L)$$

$$z_j^l = \sum_i w_{ji}^l a_i^{l-1} + b_j^l \qquad a_j^l = \sigma\left(\sum_i w_{ji}^l a_i^{l-1} + b_j^l\right) = \sigma(z_j^l)$$

$$\delta_j^L \equiv \frac{\partial E}{\partial z_j^L} = \frac{\partial E}{\partial a_i^L}\frac{\partial a_i^L}{\partial z_j^L} = \frac{\partial E}{\partial a_j^L}\sigma'(z_j^L) \qquad (1)$$

$$\delta_j^l \equiv \frac{\partial E}{\partial z_j^l} = \frac{\partial E}{\partial z_i^{l+1}}\frac{\partial z_i^{l+1}}{\partial z_j^l} = \frac{\partial z_i^{l+1}}{\partial z_j^l}\delta_i^{l+1}$$

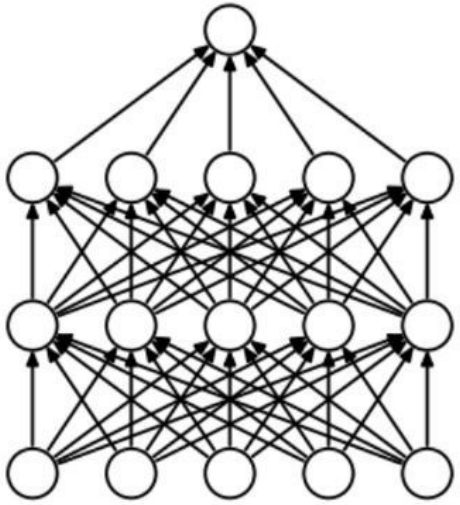
$$= \frac{\partial(\sum_i w_{ij}^{l+1} a_j^l + b_i^{l+1})}{\partial z_j^l}\delta_j^{l+1} = \sum_i w_{ij}^{l+1}\delta_i^{l+1}\sigma'(z_j^l) \qquad (2)$$

$$\frac{\partial E}{\partial w_{ji}^l} = \frac{\partial E}{\partial a_i^l}\frac{\partial a_i^l}{\partial z_j^l}\frac{\partial(w_{ji}^l a_i^{l-1})}{\partial w_{ji}^l}$$

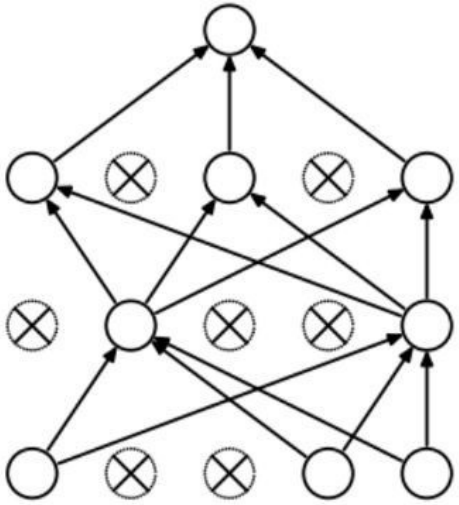
Extra Regularization for Neural Nets

Dropout: accuracy in the absence of certain information

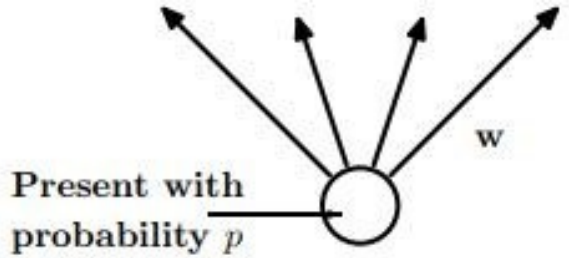
- Prevent dependence on any one (or any small combination) of neurons



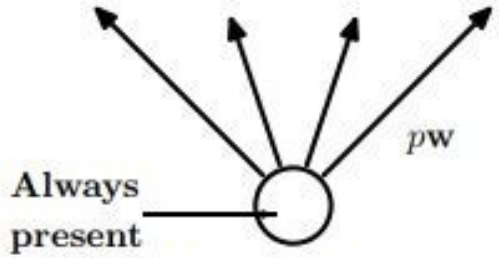
(a) Standard Neural Net



(b) After applying dropout.



(a) At training time



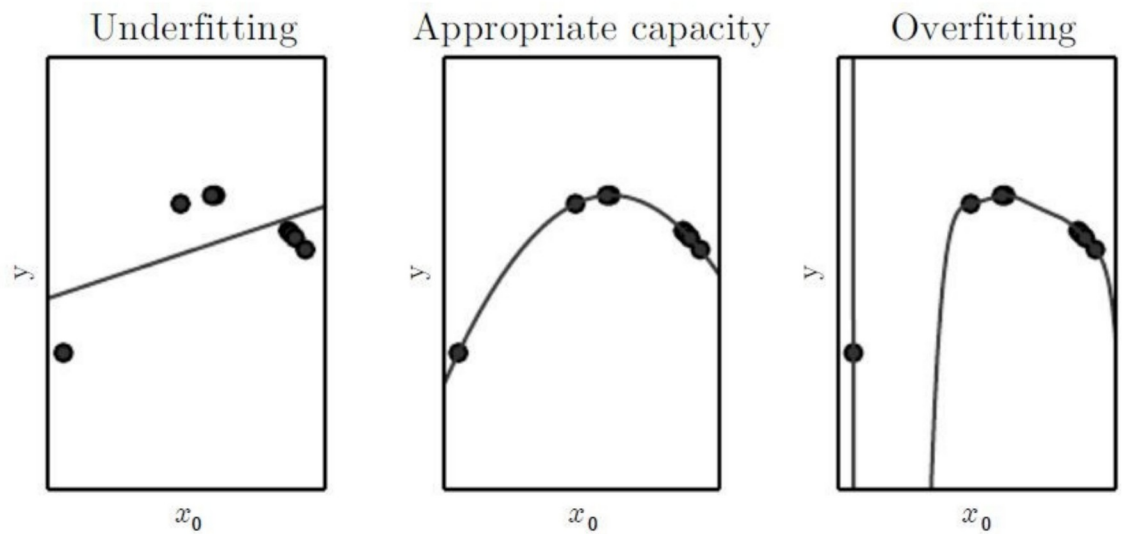
(b) At test time

Capacity, Overfitting and Underfitting

1) Make training error small
2) Make the gap between training and test error small

Low capacity ← Capacity (fitting ability) → High capacity

$\hat{y} = \sum_{i}^{1} w_i x^i$ $\hat{y} = \sum_{i}^{2} w_i x^i$ $\hat{y} = \sum_{i}^{9} w_i x^i$



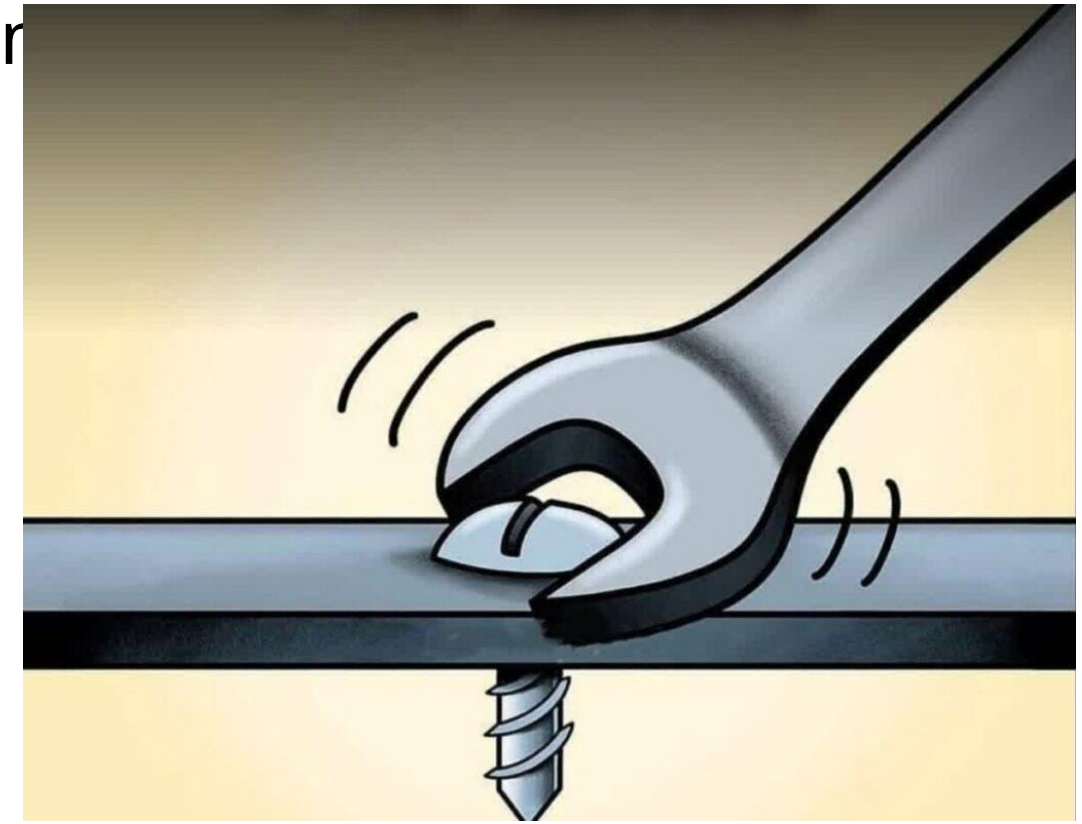
Time for a quiz and tutorial!



https://pe.app/response_links/2443bcc5-a4f2-41c6-8fd2-7f1465ef53b6/start

Back to the code

When people want to use Machine Learning without r



How training works

1. In each ***epoch***, randomly shuffle the training data
2. Partition the shuffled training data into ***mini-batches***
3. For each mini-batch, apply a single step of **gradient descent**
   - **Gradients** are calculated via ***backpropagation*** (the next topic)
4. Train for multiple epochs

Debugging a neural network

- What can we do?
  - Should we change the learning rate?
  - Should we initialize differently?
  - Do we need more training data?
  - Should we change the architecture?
  - Should we run for more epochs?
  - Are the features relevant for the problem?
- Debugging is an art
  - We'll develop good heuristics for choosing good architectures and hyper parameters

Extra readings

Deep Learning book:

- Chapter 5.9: Intro to Stochastic Gradient Descent (SGD)
- Chapter 6: Multilayer perceptrons
- Chapter 6.2.2: Output Units (Activation functions)
- Chapter 6.5: Back-Propagation
- Chapter 8.3: Basic Algorithms (Optimizers)